Fast matrix multiplication algorithms in Rust

Computational laboratory project report

Alberto Defendi
Project supervisor: Leonardo Robol

Abstract

We develop a Rust crate (library) that given matrices $A, B \in \mathbb{R}^n$ computes their product $C = AB$ using fast matrix multiplication algorithms, for any given CP decomposition.

Contents

1 Introduction

2 Notation and tensor preliminaries

The relevant notation for our work is in Table **??**. We briefly review basic tensor preliminaries, following the notation in [**?**]. A *tensor* is a multidimensional array, and in this work we deal only with 3-dimensional (or 3-way) tensors $\mathcal{X} \in \mathbb{R}^{m \times n \times p}$. The *k-th frontal slice* of $\mathcal{X}$ is the matrix $\mathbf{X}_k$. For $\mathbf{u} \in \mathbb{R}^m$, $\mathbf{v} \in \mathbb{R}^n$, $\mathbf{w} \in \mathbb{R}^p$, we define the rank-1 *outer product* tensor $\mathcal{X} = \mathbf{u} \circ \mathbf{v} \circ \mathbf{w} \in \mathbb{R}^{m \times n \times p}$ with entries $x_{ijk} = u_i v_j w_k$. Addition of tensors is defined entry-wise. The *rank* of a tensor $\mathcal{X}$ is defined as the minimum number of rank-one tensors that generate $\mathcal{X}$ as their sum. We define the *CP* decomposition of rank R of a tensor $\mathcal{X}$ as the sum $\mathcal{X} = \sum_{r=1}^{R} u_r \circ v_r \circ w_r$. We denote it by $\mathcal{X} = [\![U, V, W]\!]$, where U, V and W are matrices with R columns given by u_r, v_r, and w_r. An important operation in the context of tensors is the *mode−k product*, which is defined as. In this work, we will often use the equation $\mathcal{X} \times_1 a^\top \times_2 b^\top = c$, with $c_k = a^\top \mathbf{X}_{(k)} b = \sum_{i=1}^{m} \sum_{j=1}^{n} x_{ijk}$ to refer to matrix multiplication algorithms.

2.1 Fast algorithms as low-rank tensor decomposition

Let X, Y, Z be finite-dimensional vector spaces of dimensions I, J, K respectively. A bilinear form is a mapping $f : X \times Y \to Z$ that is linear in each entry, in other words such that $f(x, \cdot)$ and $f(\cdot, y)$ are linear for all $x \in X$ and $y \in Y$. Any bilinear form can be represented by a matrix M of coefficients: $f(x, y) = x^\top M y = \sum_i \sum_j m_{ij} x_i y_j$, and we can write it component-wise using a tensor $\mathcal{T}$ of coefficients t_{ijk} as follows:

$$f_k(x, y) = \sum_{i=1}^{I} \sum_{j=1}^{J} t_{ijk} x_i y_j, \quad \text{for all } 1 \leq k \leq K, \tag{1}$$

Table 1: Summary of notation.

$\langle M, K, N \rangle$	Bilinear form representing base case matrix multiplication of an $M \times K$ matrix by a $K \times N$ matrix.
$P \times Q \times R$	Dimensions of actual matrices that are multiplied ($P \times Q$ matrix multiplied by $Q \times R$ matrix).
$[\![\mathbf{U}, \mathbf{V}, \mathbf{W}]\!]$	Factor matrices corresponding to a tensor CP decomposition that provides a fast algorithm.
$\mathcal{X}$	Order-3 tensor.
$\mathcal{X} = \mathbf{u} \circ \mathbf{v} \circ \mathbf{w}$	Rank-1 tensor with entries $X_{ijk} = u_i v_j w_k$.
$\mathbf{X}_{(i)}$	ith frontal slice of $\mathcal{X}$, the matrix of entries $X_{:,:,i}$.
$\mathbf{a}_k$, A_{ij}	kth column and i, j entry of matrix $\mathbf{A}$.
$\text{vec}(\mathbf{A})$	Row-order vectorization of the entries of $\mathbf{A}$.
$\text{nnz}(\cdot)$	Number of non-zero entries of an object.

or in more succint tensor notation,

$$f = \mathcal{X} \times_1 x \times_2 y. \tag{2}$$

Extend and move before defining matmul formula

3 Fast matrix multiplication

Matrix multiplication is a bilinear operation; given two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, their product is

$$C = AB = \left(\sum_{k=1}^{n} A_{ik} B_{kj} \right)_{\substack{1 \le i \le m, \\ 1 \le j \le p}} = \left[A_{:1} | \cdots | A_{:n} \right] \begin{bmatrix} B_{1:} \\ \vdots \\ B_{n:} \end{bmatrix} = \sum_{k=1}^{n} A_{:k} B_{k:},$$

We indicate with $\langle m, n, p \rangle$ the bilinear form that represents the multiplication between A and B. Thus we can rewrite this form as in Equation : fixing the natural ordering on the entries of $\mathbf{A}$ and $\mathbf{B}$ and the inverse ordering on $\mathbf{C} = \mathbf{AB}$, we get a representation of $\mathcal{X} \in \mathbb{R}^{mn \times np \times mp}$ of the matrix multiplication $\langle m, n, p \rangle$ such that

$$\text{vec}(\mathbf{C}^\top) = \mathcal{X} \times_1 \text{vec}(\mathbf{A})^\top \times_2 \text{vec}(\mathbf{B})^\top. \tag{3}$$

Note that by transposing C and then vectorizing, we get a vector whose indexes are in reverse order.

Tensor definition

If we are given a CP decomposition of rank r of the tensor $\mathcal{X} = [\![U, V, W]\!]$, we can rewrite Equation **??** into

$$\text{vec}(\mathbf{C}^\top) = \sum_{l=1}^{r} (\underbrace{\mathbf{u_l}^\top \text{vec}(\mathbf{A})}_{s_l}) \cdot (\underbrace{\mathbf{v_l}^\top \text{vec}(\mathbf{B})}_{t_l}) \mathbf{w_l} = \sum_{l=1}^{r} (\underbrace{s_l \cdot t_l}_{m_l}) w_l, \tag{4}$$

where $\mathbf{u_l}$, $\mathbf{v_l}$, $\mathbf{w_l}$ denote the columns of $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$. This reduces the number of active multiplications to the rank r of $\mathcal{X}$.

3.1 The classic matrix multiplication algorithm tensor formulation

We present the $\langle 2,2,2\rangle$ classical matrix multiplication algorithm in the context of tensors. When $m,n,p>2$ we can always partition them in blocks as:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \tag{5}$$

where the sizes of the subblocks are depicted in the above equation. This algorithmic formulation is often called *divide et impera*, as it consists in breaking the problem into larger subproblems, and later recombining the results. In particular, we can view the product AB as the product between 2×2 block matrices

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11}+A_{12}B_{21} & A_{11}B_{12}+A_{12}B_{22} \\ A_{21}B_{11}+A_{22}B_{21} & A_{21}B_{12}+A_{22}B_{22} \end{bmatrix}. \tag{6}$$

The naive algorithm proceeds by combining a set of eight matrix multiplications with four additions:

$$\begin{aligned} M_1 &= A_{11}\cdot B_{11} & M_2 &= A_{12}\cdot B_{21} & M_3 &= A_{11}\cdot B_{12} \\ M_4 &= A_{12}\cdot B_{22} & M_5 &= A_{21}\cdot B_{11} & M_6 &= A_{22}\cdot B_{21} \\ M_7 &= A_{21}\cdot B_{12} & M_8 &= A_{22}\cdot B_{22} \end{aligned}$$

$$\begin{aligned} C_{11} &= M_1+M_2 & C_{12} &= M_3+M_4 \\ C_{21} &= M_5+M_6 & C_{22} &= M_7+M_8 \end{aligned}$$

The number of flops performed by standard matrix multiplication for $N\times N$ matrices is $T(N) = 8T(\frac{N}{2})+4(\frac{N}{2})^2$ for $N>1$. This can be solved by the master method to get $T(N)=2N^3-N^2$, where we assume that N is a power of two. In Section **??** we explain how to handle all dimensions.

For example, when $m=n=p=2$, we get the tensor with frontal slices

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_2 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

This yields, for example,

$$X_3 \times_1 \mathrm{vec}(A)^\top \times_2 \mathrm{vec}(B)^\top = \begin{bmatrix} a_{11} & a_{21} & a_{12} & a_{22} \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{21} \\ b_{12} \\ b_{22} \end{bmatrix} = a_{21}b_{11}+a_{22}b_{21} = c_{21}.$$

Note that the formula in Equation **??** in the context of block matrix multiplication in Equation **??** yelds

to

$$\operatorname{vec}(A) = \begin{bmatrix} A_{11} \\ A_{21} \\ A_{12} \\ A_{22} \end{bmatrix}, \quad \operatorname{vec}(B) = \begin{bmatrix} B_{11} \\ B_{21} \\ B_{12} \\ B_{22} \end{bmatrix}, \quad \operatorname{vec}(C^\top) = \begin{bmatrix} C_{11} \\ C_{12} \\ C_{21} \\ C_{22} \end{bmatrix}.$$

Also the factors s_l and t_l in Equation **??** become

$$s_l = u_l^\top \operatorname{vec}(A) = \sum_{i=1}^{4} U_{li} \operatorname{vec}(A)_i, \quad t_l = V_l^\top \operatorname{vec}(B) = \sum_{i=1}^{4} V_{li} \operatorname{vec}(B)_i$$

3.2 Strassen algorithm using CP

The idea of fast matrix multiplication algorithms is to perform fewer recursive matrix multiplications at the expense of more matrix additions. The most well known fast algorithm is due to Strassen, and follows the same block structure:

$$\begin{aligned}
&S_1 = A_{11} + A_{22} && S_2 = A_{21} + A_{22} && S_3 = A_{11} \\
&S_4 = A_{22} && S_5 = A_{11} + A_{12} && S_6 = A_{21} - A_{11} \\
&S_7 = A_{12} - A_{22} \\
&T_1 = B_{11} + B_{22} && T_2 = B_{11} && T_3 = B_{12} - B_{22} \\
&T_4 = B_{21} - B_{11} && T_5 = B_{22} && T_6 = B_{11} + B_{12} \\
&T_7 = B_{21} + B_{22}
\end{aligned}$$

$$\begin{aligned}
&M_r = S_r T_r, \quad 1 \le r \le 7 \\
&C_{11} = M_1 + M_4 - M_5 + M_7 && C_{12} = M_3 + M_5 \\
&C_{21} = M_2 + M_4 && C_{22} = M_1 - M_2 + M_3 + M_6
\end{aligned}$$

This algorithm uses 7 matrix multiplications and 18 matrix additions. The number of flops performed by the algorithm is $T(N) = 7T(\frac{N}{2}) + 18(\frac{N}{2})^2$ for $N > 1$, solving the recursion the complexity is $T(N) = 7N^{\log_2 7} - 6N^2 = O(N^{\log_2 7}) = O(N^{2.81})$.

complete

There are natural extensions to Strassen algorithm:

1. We can't use less than 7 number of additions for $\langle 2, 2, 2 \rangle$.
2. Reduce the number of addition from 18 down to 15, which leads to Strassen-Winograd algorithm . cite
3. We can change constant on the leading term by chosing a bigger base case dimension (and using the standard algorithm (also called *gemm*) on the base case).
4. We can use blocking schemes different from $\langle 2, 2, 2 \rangle$.

4 Implementation and practical consideration

4.1 Technical overview

Similarities: - Both use MKL for base, but Rust project also uses Faer. In the C++ project, MKL is imported, while in Rust it is binded via FFI.

Differences - Numerical computations in the c++ are based on Lapack with OpenMP parallelization, while - C++ algorithm are code-generated by a script that parses CP tensor decomposition slices and translates the matrix multiplication expression into C++ code, while Rust loads the tensor at runtime [1] and then applies the pseudocode in Algorithm **??**. The advantage of the first is speed, at the cost of more complex code and parallelism. The advantage of the second is simplicity of code, at the cost of speed. - C++ project

fix

The implementation consists in a Rust porting of the c++ project made by Benson and Ballard summarized in [**?**], available in the relative repository under BSD 2-Clause. They provide a fast and efficient framework for benchmarking parallel and sequential fast matrix multiplication algorithms, achieved trough code generation, using lapack and MKL dgemm for numerical computation, and OpenMP for parallelization. Our project differs from their implementation mainly for the language (Rust) and code generation—we apply CP matrix multiplication without code generation, which delivers simpler code at the cost of speed . Also, our project uses the numerical library Faer (cite) with Rust's Rayon parallelization, and selectively switches to Faer or MKL Dgemm for matrix multiplication, while the other uses Lapack and MKL Dgemm, and OpenMP for parallelization.

is this True?

Following the work [**?**] we implement a fast multiplication algorithm based on CP decomposition given the U, V and W matrices representing the algorithm.

Fast matrix multiplication algorithm such as Strassen face two significant problems in practical implementation, which are recursion cutoff and input matrices sizes not matching with base case. The former can solved by …and the latter by dynamic peeling **??**.

4.2 Algorithm schema

We now discuss the algorithm schema for Strassen algorithm for clarity, though results generalize easily for any $\langle m, n, p \rangle$ base case. Algorithm **??** shows our implementation of Strassen's algorithm, which easily extends to other CP algorithms by changing base conditions and block sizes—this is how it is implemented in practice.

4.3 Execution platform and reproducibility

Since the comparison targets both sequential and parallel matrix multiplication kernels, we ran all experiments on one node of a dual-socket AMD EPYC 7763 server (64 physical cores per socket, 2 threads per core) with 2 TB of RAM. The system ran Ubuntu 24.04.5 LTS with Linux 5.15.0-179-generic on x86_64. For sequential benchmarks, library-level threading was restricted with `OMP_NUM_THREADS=1` and

[1] when program is launched, before doing the multiplication, which does not affect speed since the matrices decomposition is cached at runtime

Algorithm 1 Strassen algorithm using CP factorization.

procedure STRASSEN(A, B, N)
  if STOP-RECURSING is True **then**
    return $A \cdot B$
  end if
  if $N = 2$ **then**
    STRASSEN-BASE-MATMUL(A, B)
  end if
  if $N \mod 2 > 0$ **then**
    DYNAMIC-PEELING(A, B, N) ▷ Recursive call on peeled blocks.
  end if
  Compute $\text{vec}(A)$ and $\text{vec}(B)$ with block size $\frac{N}{2}$.
  for $r = 1, \ldots, R$ **do**
    $S_r \leftarrow \sum_{i=1}^{4} U_{i,r} \text{vec}(A)_i$
    $T_r \leftarrow \sum_{i=1}^{4} V_{i,r} \text{vec}(B)_i$
    $M_r \leftarrow$ STRASSEN$(S_r, T_r, \frac{N}{2})$ ▷ Recursive call.
  end for
  for $r = 1, \ldots, R$ **do**
    for $i = 1, \ldots, 2$ **do**
      for $j = 1, \ldots, 2$ **do**
        $w \leftarrow W_{\mathbb{L}^*(i,j)}, r = W_{2i+j,r}$
        if $w \neq 0$ **then**
          $C_{i \cdot \frac{N}{2}:(i+1) \cdot \frac{N}{2}, j \cdot \frac{N}{2}:(j+1)\frac{N}{2}} \leftarrow w \cdot M_r$
        end if
      end for
    end for
  end for
  return C.
end procedure

`MKL_NUM_THREADS=1`. These choices remove scheduler migration and library-level parallelism from the measurements to isolate microarchitectural performance and compiler optimizations without the interference of thread scheduling and synchronization overheads. The software environment is configured and reproduced using Environment Modules. The configuration loads `gcc/13.2.0` as the base compiler toolchain, alongside the `intel-oneapi-compilers` version `2025.0.4-gcc-11.4.0`. To provide a highly optimized BLAS/LAPACK backend, the environment loads `intel-oneapi-mkl/2024.2.2-intel-oneapi-mpi-2021.14.1-oneapi-2025.0.4`. When building the C wrappers and Rust bindings, all compilation targets are explicitly optimized for the server architecture using the target microarchitecture flags (`-march=znver3 -mtune=znver3`). In the C++ project, to support both sequential and parallel modes within a single executable, the MKL backend was dynamically linked against the GNU OpenMP threading layer (`mkl_gnu_thread` and `-lgomp`) instead of the sequential MKL library. We build the Rust code with the nightly-2026-06-23 toolchain because this release uses LLVM 22. Benchmark runs set `RUSTFLAGS="-C target-cpu=znver3"`, generating optimized vector instructions leveraging AVX2 and FMA. Since the C/C++ wrappers and Rust code compile with matching CPU targets, a custom `build.rs` script compiles the Intel MKL FFI C wrapper (`mkl.c`) using `gcc` with `-O3 -march=znver3 -mtune=znver3`. During linking, `build.rs` ensures that the Intel MKL dynamic libraries are correctly loaded. To eliminate the need for manual `LD_LIBRARY_PATH` configuration at runtime, the MKL library path is compiled into the executable binaries as a runtime.

Here comparison with Ballard et Al (used for comparison)

4.3.1 FFI bindings

4.4 Recursive cutoff strategies

We implement two strategies for stopping recursion before calling the vendor library, which are by recursion depth and by matrices size. The former stops when a fixed number L of recursion levels is reached, and the latter when matrices sizes reaches a given size.

Cutoff comparisons in sequential case??

4.5 Dynamic Peeling

In Strassen algorithm we want to multiply $\langle m, n, p \rangle$ matrices by splitting in four blocks as in Equation **??**. If m, n and p are even, then it is not possible to split blocks in even sizes. The easiest fix for this is padding the matrix with zeros until an even size is reached, but this adds a significant memory waste. A more efficient approach, called *dynamic peeling*, consists in stripping the extra row off and/or column and adding their contribution to the final results. In detail,

For the matrix multiplication $C = AB$, where A is an $m \times n$ matrix and B is an $n \times p$ matrix. The matrix A is split into an $(m-1) \times (n-1)$ core matrix A_{11}, alongside its peeled boundary vectors a_{12}, a_{21}, and the scalar corner element a_{22}. Similarly, the matrix B is split into an $(n-1) \times (p-1)$ core matrix B_{11}, with its corresponding peeled components b_{12}, b_{21}, and b_{22}:

$$A = \left[\begin{array}{cc} A_{11} & a_{12} \\ \hline a_{21} & a_{22} \end{array}\right], \qquad B = \left[\begin{array}{cc} B_{11} & b_{12} \\ \hline b_{21} & b_{22} \end{array}\right].$$

The final block matrix product is computed directly through a combination of the core Strassen multiplication ($A_{11}B_{11}$) and low-rank vector/scalar fixup modifications:

$$\left[\begin{array}{cc} C_{11} & c_{12} \\ \hline c_{21} & c_{22} \end{array}\right] = \left[\begin{array}{cc} A_{11}B_{11} + a_{12}b_{21} & A_{11}b_{12} + a_{12}b_{22} \\ \hline a_{21}B_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{array}\right].$$

Here, only the sub-product $A_{11}B_{11}$ is executed recursively using Strassen's matrix multiplication, while all other terms are computed via standard matrix multiplication.

5 Parallel algorithms for shared memory

Matrix multiplication is a problem subject to both computational and memory bound. The first is limited by the amount of computation the device can handle (i.e cores,cpu/gpu speed), while the latter by physical storage of the matrices (i.e available RAM memory). In a parallel implemntation, when dealing with recursive algorithms which use divide-et-impera (like Strassen), it is important in practice to balance

algorithmic load between threads to avoid overhead caused by both taking recursive branches and doing parallel matrix multiplication. To address this issue, we adopt the scheduling schema suggested by Benson and Ballard in their work [**?**].

6 Finding the optimal recursive cutoff

In divide et impera algorithms such as Strassen, we can't afford taking many recursive stes, as spawning many children constitutes a significant recursive overhead. Thus, finding an optimal cutoff is important for stopping the recursion and switch to standard gemm. We try two techniques: the first is breaking by size, that is when the sizes of matrices in the recursion fall under a certain size, the latter is taking a fixed number L of recursions. The authors in [**?**] explain an euristical process for finding the optimal cutoff size, which can be resumed in looking at the performance plot of the gemm function, where (figure) the function exibits a ramp-up and then flatten for large dimensions. They take a recursion step only if the recursive sub-problem fall in the flat part of the curve. This is done manually by the authors. During the experimentation we developed a function to plot this curve, and its derivative (using splines) to automatically find this size. In practice, we never use this as we believe that our code is not efficient enough to implement this strategy.

6.1 Implementation

We use Rust's Rayon to handle parallelization, and we can summarize the parallel scheduling as following:

- **BFS:** Each recursive matrix multiplication routine and the associated matrix additions are launched by Rayon as an iterable, and then are collected in an array of matrices $[M_1, \ldots, M_r]$.
- **DFS:** Each vendor matrix multiplication call uses all thread, and matrix multiplications are always fully parallelized.
- **Hybrid:** The Hybrid approach developed in [**?**] compensates the for the load imbalance in BFS by applying DFS on a subset of the base case problems. With L levels of recursion and P threads, the hybrid algorithm applies task parallelism (BFS) to the first $R^L - (R^L \mod P)$ multiplication. Because the number of subproblems is a multiple of P, this part is load balanced. On the remaining $R^L \mod P$ subproblems, all threads are used on each matrix multiplication (DFS).

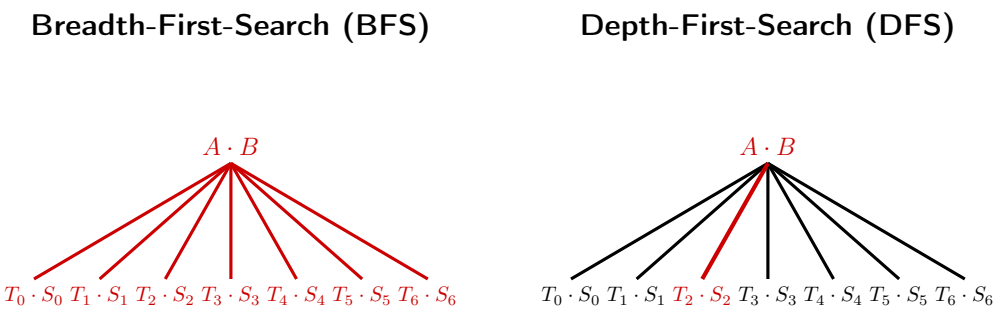

Figure 1: BFS and DFS load balancing. The parallelized tasks are in red.

7 Experimental results

7.1 Metrics

All benchmarks are recorded using Rust's system clock, performing warm-up before doing measuring each matrix multiplication. In order to compare the performance of matrix multiplication algorithms with different ocmputational costs, we use the *effective* GFLOPS metric for $\langle m, n, r \rangle$ matrix multiplication:

$$\text{effective GFLOPS} = \frac{2mnp - mp}{\text{time in seconds} \cdot 10^9}, \tag{7}$$

where the numerator can be rewritten as $2mnp - mp = mnp + m(n-1)p$, which is the number of multiplications and additions performed for classical matrix multiplication. This allows us to compare all of the algorithms on an inverse-time scale, normalized by problem size [**?**]. For fast algorithms, considering both multiplications and additions in the metric is more accurate than taking multiplications only [**?**].

7.2 Measuring impact of base gemm

We aim to understand how the choice of the base matrix multiplication impacts performance of fast algorithms. In our experiments, we use random square matrices with real entries of sizes $N \in \{2, 4, \ldots, 2^k\}$. We are interested in finding the N such that using a fast matrix algorithm is faster than doing standard matrix multiplication.

7.3 Comparison of MKL dgemm FFI and Faer matmul

In the project we have two different matrix multiplication routines: MKL dgemm and faer base matmul.

- **Faer matmul:** The method is entirely written in Rust and is part of the faer library.
- **MKL dgemm:** is a routine in the Intel Math Kernel Library (MKL) for multiplying double precision matrices. In Rust, this routine runs via a Foreign Function Interface (FFI) calling the C wrapper `cblas_dgemm`. In the C++ project, the function is called directly. Both projects use the same library version on the architecture.

From Figure **??**, we see that all functions have a ramp-up phase and then flatten for larger sizes. Also, dgemm performs better than faer for larger sizes, and we believe this is due the many low-level optimizations in dgemm. Further, there is a discrepancy between Rust FFI dgemm and the C++ code which tends to decrease for larger size. We think that this might be due the type conversion which is performed during the binding.

7.4 Numerical comparison with Benson&Ballard Strassen

We perform a numerical performance comparison between our optimized Rust implementation of Strassen's algorithm and the reference C++ implementation by Benson and Ballard [**?**]. The results are shown in Figure **??**. We compare the algorithms across one, two, and three levels of recursion using different thread scheduling paradigms: Sequential, Parallel DFS, Parallel BFS, and Parallel Hybrid.

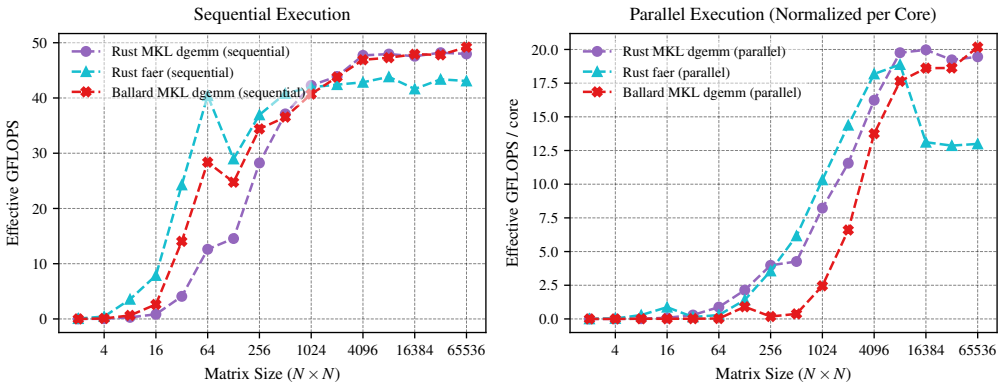

Figure 2: Comparison between MKL and faer in sequenziale and parallel execution.

8 Further work

- **Further code optimization:** We believe that code can be made more efficient by doing a more fine refactory with profiling result.
- **Gemm comparison:** Though we choose mkl-dgemm for a better comparison, our ffi bindings easily extend to any other gemm, such as OpenBlas, which we leave to further work.
- **Gpu/distributed version:** We would like to port the project to GPU or distributed computing to extend results with higher parallelism and test larger sizes.

9 Acknowledgments

We acknowledge the authors of [**?**] for providing instructions for linking MKL with Rust, and the department of Mathematics of the University of Pisa for offering access to their HPC cluster.

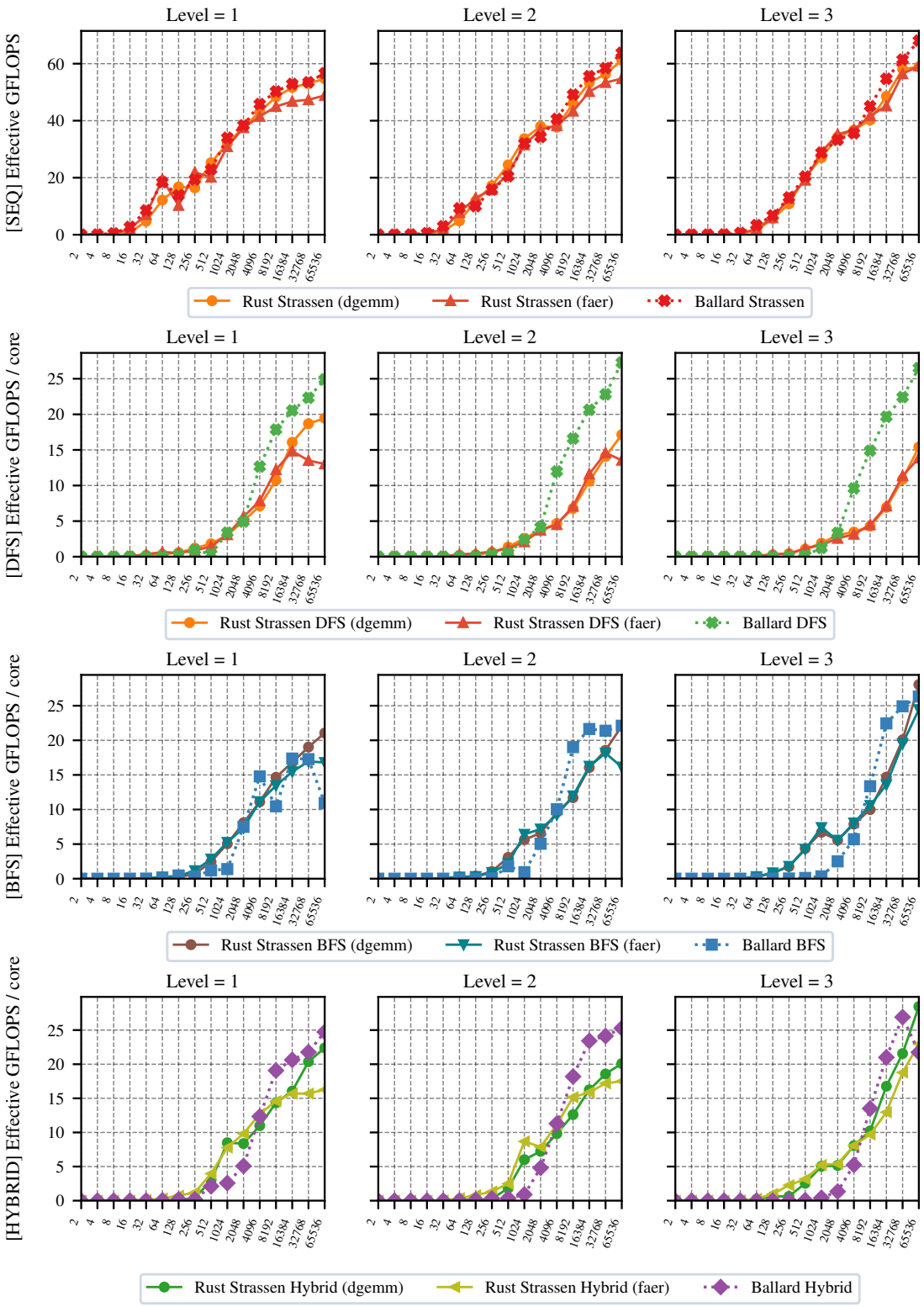

Figure 3: Performance comparison of our Rust Strassen implementation (using dgemm and faer base cases) against the Benson & Ballard reference benchmarks across sequential and parallel execution modes.

References

[1] Grey Ballard and Tamara G. Kolda. *Tensor Decompositions for Data Science*. Cambridge University Press, 2025.

[2] Austin R. Benson and Grey Ballard. A framework for practical parallel fast matrix multiplication. In *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP 2015, page 42–53, New York, NY, USA, 2015. Association for Computing Machinery.

[3] Benjamin Lipshitz, Grey Ballard, James Demmel, and Oded Schwartz. Communication-avoiding parallel strassen: Implementation and performance. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–11, 2012.

[4] Luca Lombardo and Fabio Durastante. Evaluating rust for sparse matrix kernels in scientific computing, 2026.

[5] S. Huss-Lederman, E.M. Jacobson, J.R. Johnson, A. Tsao, and T. Turnbull. Implementation of strassen's algorithm for matrix multiplication. In *Supercomputing '96:Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*, pages 32–32, 1996.